

# Hash Storage

## Deployment Guide

Content-Addressable Storage · Flask + PostgreSQL + S3 · Docker Compose  
Version 1.0 · 2026-04-25

| Layer     | Technology                                  | Note                             |
|-----------|---------------------------------------------|----------------------------------|
| Frontend  | React 19 + TypeScript + Vite + Tailwind CSS | Served as static build via Nginx |
| Backend   | Python 3.12 + Flask 3 + Gunicorn            | REST API                         |
| Database  | PostgreSQL (Alpine)                         | Metadata + VFS tree              |
| Storage   | AWS S3 (prod) / MinIO (dev)                 | Binary objects                   |
| Proxy     | Nginx (Alpine)                              | HTTPS + reverse proxy            |
| Container | Docker + Docker Compose                     | Separate dev / prod configs      |

# 1. System Architecture

Hash Storage is a file management system built on the **CAS (Content-Addressable Storage)** principle. Identical file content is stored exactly once in S3, keyed by its SHA-256 hash. The PostgreSQL **fs\_nodes** tree provides the virtual filesystem (VFS) layer.

| Component                        | Role                                                      | Port               |
|----------------------------------|-----------------------------------------------------------|--------------------|
| <a href="#">Nginx</a>            | HTTPS termination, /api/ proxy, React static file serving | 80, 443            |
| <a href="#">Flask (Gunicorn)</a> | REST API — upload / download / delete / folder creation   | 5000<br>(internal) |
| <a href="#">PostgreSQL</a>       | file_blobs (CAS layer) + fs_nodes (VFS tree)              | 5432<br>(internal) |
| <a href="#">MinIO (dev)</a>      | Local S3-compatible object storage                        | 9000, 9001         |
| <a href="#">AWS S3 (prod)</a>    | Production binary storage — IAM Role auth                 | HTTPS              |

## Data Model

|                   |                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>file_blobs</b> | hash_id (PK · SHA-256), size_bytes, mime_type, s3_key, ref_count, created_at                                    |
| <b>fs_nodes</b>   | node_id (UUID PK), parent_id (FK self-ref), node_type (file folder), name, hash_id (FK), created_at, updated_at |
| <b>Indexes</b>    | idx_fs_nodes_parent_id, idx_fs_nodes_hash_id, idx_file_blobs_ref_count, uq_fs_nodes_parent_name (UNIQUE)        |

## API Endpoints

|                                |                                                               |
|--------------------------------|---------------------------------------------------------------|
| <b>POST /files/upload</b>      | Upload file — client SHA-256 hash checked; dedup or store new |
| <b>GET /nodes</b>              | List children of a folder (parent_id query param)             |
| <b>GET /nodes/:id/download</b> | Stream file download from S3                                  |
| <b>DELETE /nodes/:id</b>       | Delete node — decrement ref_count; GC blob when ref_count = 0 |
| <b>POST /folders</b>           | Create a new folder                                           |
| <b>GET /health</b>             | Check DB + S3 connectivity                                    |

# 2. Prerequisites

## Local Development

|                |                                                       |
|----------------|-------------------------------------------------------|
| Docker Desktop | 4.x or later (Docker Engine 24+, Compose v2 included) |
| Node.js        | 22.x LTS                                              |
| Python         | 3.12+                                                 |
| Git            | 2.x or later                                          |

## AWS Production Environment

|              |                                                                                         |
|--------------|-----------------------------------------------------------------------------------------|
| EC2 Instance | t4g.medium recommended — 2 vCPU / 4 GB RAM / ARM / ~\$24/mo — all-in-one Docker Compose |
| S3 Bucket    | Must be created in advance · Block all public access · Region: ap-northeast-2           |
| IAM Role     | Attach to EC2 · AmazonS3FullAccess or custom bucket policy (least privilege)            |
| Domain       | Route 53 or external DNS — A record pointing to EC2 public IP                           |
| OS           | Ubuntu 24.04 LTS recommended                                                            |

## 3. Local Development Setup

### Step 1 — Clone Repository & Configure Environment

```
git clone https://github.com//hash_storage.git
cd hash_storage
cp .env.example .env # default values work out of the box
```

### Step 2 — Start Containers

```
./scripts/dev-up.sh
# or directly:
docker compose up -d --build
```

#### Service URLs

|                 |                                                 |
|-----------------|-------------------------------------------------|
| Frontend (Vite) | http://localhost:5173                           |
| Backend (Flask) | http://localhost:5000                           |
| MinIO Console   | http://localhost:9001 (minioadmin / minioadmin) |
| PostgreSQL      | localhost:5432 (fms_user / fms_password)        |

### Step 3 — Seed Test Data (optional)

```
python scripts/seed_fake_nodes.py # insert 100k+ virtual nodes
python scripts/benchmark_tree.py # measure B-Tree traversal performance
```

### Step 4 — Stop Containers

```
./scripts/dev-down.sh # keep volumes (data preserved)
docker compose down -v # also remove volumes (full reset)
```

#### Verifying Deduplication

Upload the same file to two different folders. S3 stores it only once; the DB **ref\_count** increments. The frontend banner confirms: *"Duplicate detected — linked without re-uploading."*

## 4. AWS Production Deployment

---

### Step 1 — Create S3 Bucket

```
aws s3 mb s3://your-fms-bucket --region ap-northeast-2
aws s3api put-public-access-block --bucket your-fms-bucket \
--public-access-block-configuration \
BlockPublicAcls=true,IgnorePublicAcls=true,\
BlockPublicPolicy=true,RestrictPublicBuckets=true
```

### Step 2 — IAM Role (Least-Privilege Policy)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::your-fms-bucket",
      "arn:aws:s3:::your-fms-bucket/*"
    ]
  }]
}
```

### Step 3 — EC2 Initial Setup (Ubuntu 24.04)

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y docker.io docker-compose-plugin git
sudo usermod -aG docker $USER && newgrp docker
```

### Step 4 — Configure Environment Variables

Copy `.env.example` to `.env` and set production values:

```
POSTGRES_PASSWORD=
S3_ENDPOINT_URL= # leave empty to use AWS default endpoint
S3_BUCKET_NAME=your-fms-bucket
S3_REGION=ap-northeast-2
S3_USE_SSL=true
DOMAIN=yourdomain.com
CORS_ORIGINS=https://yourdomain.com
GUNICORN_WORKERS=3
```

### Step 5 — Issue HTTPS Certificate (Let's Encrypt)

```
sudo apt-get install -y certbot
# Run while Docker containers are stopped
sudo certbot certonly --standalone -d yourdomain.com
```

### Step 6 — Start Production Containers

```
docker compose -f docker-compose.prod.yml up -d --build
curl https://yourdomain.com/api/health # expected: {"status":"ok"}
```

## Certificate Auto-Renewal (cron)

```
# /etc/cron.d/certbot-renew
0 3 * * 1 root certbot renew --quiet && \
docker compose -f /home/ubuntu/hash_storage/docker-compose.prod.yml restart nginx
```

# 5. Environment Variable Reference

## Backend (.env)

|                   |                                                        |
|-------------------|--------------------------------------------------------|
| POSTGRES_DB       | Database name (default: fms)                           |
| POSTGRES_USER     | PostgreSQL user (default: fms_user)                    |
| POSTGRES_PASSWORD | DB password — use a strong value in production         |
| S3_ENDPOINT_URL   | MinIO: http://minio:9000   AWS: leave empty            |
| S3_ACCESS_KEY     | MinIO: minioadmin   AWS IAM Role: leave empty          |
| S3_SECRET_KEY     | MinIO: minioadmin   AWS IAM Role: leave empty          |
| S3_BUCKET_NAME    | Bucket name (default: fms-bucket)                      |
| S3_REGION         | AWS region (default: ap-northeast-2)                   |
| S3_USE_SSL        | true (AWS S3)   false (MinIO)                          |
| FLASK_ENV         | development   production                               |
| FLASK_DEBUG       | 1 (dev)   0 (prod)                                     |
| CORS_ORIGINS      | * (dev)   https://yourdomain.com (prod)                |
| GUNICORN_WORKERS  | Recommended: (vCPU x 2) + 1 (default: 3)               |
| DOMAIN            | Production domain — used in Nginx SSL certificate path |

## Frontend (Vite)

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| VITE_API_BASE_URL | Dev: http://localhost:5000   Prod (Nginx proxy): /api |
|-------------------|-------------------------------------------------------|

## 6. Nginx Routing

The production Nginx container (infra/nginx/prod.conf) handles HTTPS termination, React static file serving, and backend API proxying in a single container.

```
# HTTP -> HTTPS redirect
server {
    listen 80; server_name ${DOMAIN};
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

# HTTPS main server
server {
    listen 443 ssl;
    ssl_certificate /etc/letsencrypt/live/${DOMAIN}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/${DOMAIN}/privkey.pem;
    client_max_body_size 100M;
}

# React SPA (frontend container)
location / { proxy_pass http://frontend:80; }

# Flask REST API (strip /api/ prefix before forwarding)
location /api/ {
    rewrite ^/api/(.*)$ /$1 break;
    proxy_pass http://backend:5000;
    proxy_read_timeout 120s;
}
}
```

The default upload size limit is **100 MB** (client\_max\_body\_size 100M). Gunicorn is configured with --timeout 120 to match, preventing connection drops on large file uploads.



# 7. Operations

## Container Management

|                   |                                                                        |
|-------------------|------------------------------------------------------------------------|
| Check status      | <code>docker compose -f docker-compose.prod.yml ps</code>              |
| All logs          | <code>docker compose -f docker-compose.prod.yml logs -f</code>         |
| Backend logs only | <code>docker compose -f docker-compose.prod.yml logs -f backend</code> |
| Restart service   | <code>docker compose -f docker-compose.prod.yml restart backend</code> |
| Health check      | <code>curl https://yourdomain.com/api/health</code>                    |

## DB Backup & Restore

```
# Backup
docker exec fms-postgres pg_dump -U fms_user fms > backup_$(date +%Y%m%d).sql

# Restore
cat backup_YYYYMMDD.sql | docker exec -i fms-postgres psql -U fms_user fms
```

## Code Update (zero-downtime redeploy)

```
git pull origin main
docker compose -f docker-compose.prod.yml up -d --build
# If schema changed:
docker exec fms-backend flask db upgrade
```

## Orphaned Blob Check

```
-- Blobs with ref_count = 0 indicate a DB/S3 inconsistency
SELECT hash_id, s3_key, ref_count FROM file_blobs WHERE ref_count = 0;
```

## 8. Troubleshooting

| Symptom                        | Cause                                  | Fix                                                                |
|--------------------------------|----------------------------------------|--------------------------------------------------------------------|
| Backend fails to connect to DB | Started before DB healthcheck passed   | Verify depends_on condition: service_healthy in docker-compose.yml |
| MinIO bucket not found         | minio-init service failed              | docker compose logs minio-init                                     |
| Upload returns 413             | Nginx size limit exceeded              | Increase client_max_body_size in prod.conf                         |
| SSL certificate error          | Certificate path mismatch              | Check /etc/letsencrypt/live/DOMAIN/ vs prod.conf                   |
| S3 AccessDenied (prod)         | IAM Role not attached or missing perms | Verify EC2 IAM Role has S3 bucket policy                           |
| Bucket creation fails (region) | Missing LocationConstraint             | storage.py handles this automatically. Verify S3_REGION            |
| CORS error in browser          | CORS_ORIGINS not set                   | Set CORS_ORIGINS=https://yourdomain.com in .env                    |
| Large upload times out         | Gunicorn / Nginx timeout               | Check GUNICORN_WORKERS and proxy_read_timeout values               |

## 9. Extension Roadmap

After the initial MVP deployment, the following phases extend the service.

| Phase   | Scope                                                                                 | Est.    |
|---------|---------------------------------------------------------------------------------------|---------|
| Phase 1 | AWS production deploy · GitHub Actions CI/CD · CloudWatch monitoring                  | 1-2 wks |
| Phase 2 | Google/GitHub OAuth 2.0 · JWT sessions · per-user fs_nodes isolation · storage quotas | 2-3 wks |
| Phase 3 | E2EE share links · Web Crypto API AES-GCM · URL fragment key delivery                 | 3-5 wks |
| Phase 4 | Online viewer (PDF.js, images) · inline text editing · Yjs real-time collaboration    | 4-8 wks |
| Phase 5 | Nginx wildcard TLS · per-user subdomains · Cloudflare DNS API auto-provisioning       | 1 wk    |

### E2EE vs. CAS Trade-off

**E2EE and server-side deduplication conflict.** Encrypted files produce different hashes even for identical content, breaking dedup. Planned approach: keep dedup for personal space; create a separate encrypted copy only when sharing.

### Estimated Monthly Cost (MVP baseline)

|                   |                                                                   |
|-------------------|-------------------------------------------------------------------|
| EC2 t4g.medium    | ~\$24/mo — 2 vCPU · 4 GB RAM · ARM · Seoul region                 |
| AWS S3            | ~\$0.023/GB/mo storage + \$0.09/GB outbound transfer              |
| Initial total     | ~\$25-35/mo (PostgreSQL runs in Docker, no RDS)                   |
| After user growth | ~\$40/mo — EC2 t4g.medium + RDS db.t4g.micro (~\$13/mo) separated |

# 10. Quick Reference Card

---

## Development Cheatsheet

```
./scripts/dev-up.sh # start all containers
./scripts/dev-down.sh # stop (data preserved)
docker compose logs -f [backend|frontend|postgres|minio] # stream logs
docker exec -it fms-postgres psql -U fms_user -d fms # open DB shell
curl http://localhost:5000/health # health check
```

## Production Cheatsheet

```
docker compose -f docker-compose.prod.yml up -d --build # deploy
git pull && docker compose -f docker-compose.prod.yml up -d --build # redeploy
docker exec fms-postgres pg_dump -U fms_user fms > backup_$(date +%Y%m%d).sql
sudo certbot renew && \
docker compose -f docker-compose.prod.yml restart nginx
docker exec fms-postgres psql -U fms_user -d fms \
-c "SELECT hash_id, ref_count FROM file_blobs WHERE ref_count = 0;"
```